Ex 8: Deploying a Fine-Tuned Language Model for

Customer Support with Evaluation (BLEU & ROUGE)

AIM:

Develop a Python-based application to fine-tune and deploy a language model for customer support using platforms like Hugging Face Spaces or Streamlit, and evaluate its performance on the entire test dataset using BLEU and ROUGE metrics.

Steps:

1. **Import Required Libraries**
   Import Transformers, Datasets, Torch, and evaluation libraries (BLEU, ROUGE).
2. **Load Dataset**
   Use a Customer Service dataset or Bank FAQ dataset containing queries and responses.
3. **Data Preprocessing**
   - Clean and normalize text
   - Prepare input-output pairs (query → response)
4. **Split Dataset**
   Divide into training and testing datasets.
5. **Load Pre-trained Model**
   Load models such as T5 or BART from Hugging Face.
6. **Fine-Tune the Model**
   Train the model using the training dataset.
7. **Save the Fine-Tuned Model**
   Store trained model and tokenizer.
8. **Build Inference Pipeline**
   Create a function to generate responses for input queries.
9. **Generate Predictions on Test Dataset**
   - Pass all test queries to the model
   - Generate responses for each query
10. **Compute BLEU Scores (BLEU-1 to BLEU-4)**

- Measure n-gram precision between generated and reference responses
- Compute BLEU-1, BLEU-2, BLEU-3, and BLEU-4 for each sample

11. **Compute ROUGE Scores**

- ROUGE-1 → unigram recall
- ROUGE-2 → bigram recall
- ROUGE-L for sequence similarity

12. **Aggregate Evaluation Metrics**

- Calculate average scores across all test samples
- Prepare overall performance summary

13. **Display Evaluation Results**

Metric	**Score**
BLEU-1	
BLEU-2	
BLEU-3	
BLEU-4	
ROUGE-1	
ROUGE-2	
ROUGE-L	

14. **Develop User Interface**

- Use Streamlit for web app
- OR deploy using Hugging Face Spaces

15. **Deploy the Application**

- Host the model for real-time customer query handling

16. **Test with Sample Queries**
    Validate the deployed system with real-time user inputs.

17. **Display Output**
    Show generated responses in the interface.

CODE :

```
!pip install transformers datasets nltk rouge_score -q

import numpy as np
import pandas as pd
import torch
from datasets import load_dataset
from transformers import T5Tokenizer, T5ForConditionalGeneration
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from rouge_score import rouge_scorer
import nltk
nltk.download('punkt')

from google.colab import files
uploaded = files.upload()

dataset = load_dataset("csv", data_files=list(uploaded.keys())[0])
dataset = dataset['train'].train_test_split(test_size=0.2)

train_data = dataset['train']
test_data = dataset['test']

print(train_data.column_names)

input_col = train_data.column_names[0]
output_col = train_data.column_names[1]

tokenizer = T5Tokenizer.from_pretrained("t5-small")
model = T5ForConditionalGeneration.from_pretrained("t5-small")

def generate_response(query):
    input_text = "question: " + str(query)
    inputs = tokenizer(input_text, return_tensors="pt", truncation=True, max_length=64)
    outputs = model.generate(**inputs, max_length=64)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

smooth = SmoothingFunction().method1

bleu1, bleu2, bleu3, bleu4 = [], [], [], []
rouge1, rouge2, rougel = [], [], []

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)

for sample in test_data:
    query = sample[input_col]
    reference = str(sample[output_col])
    prediction = generate_response(query)

    ref_tokens = [reference.split()]
    pred_tokens = prediction.split()
```

```
    b1 = sentence_bleu(ref_tokens, pred_tokens, weights=(1,0,0,0),
smoothing_function=smooth)
    b2 = sentence_bleu(ref_tokens, pred_tokens, weights=(0.5,0.5,0,0),
smoothing_function=smooth)
    b3 = sentence_bleu(ref_tokens, pred_tokens, weights=(0.33,0.33,0.33,0),
smoothing_function=smooth)
    b4 = sentence_bleu(ref_tokens, pred_tokens, weights=(0.25,0.25,0.25,0.25),
smoothing_function=smooth)

    bleu1.append(b1)
    bleu2.append(b2)
    bleu3.append(b3)
    bleu4.append(b4)

    scores = scorer.score(reference, prediction)
    rouge1.append(scores['rouge1'].fmeasure)
    rouge2.append(scores['rouge2'].fmeasure)
    rougel.append(scores['rougeL'].fmeasure)

    print("Query:", query)
    print("Reference:", reference)
    print("Prediction:", prediction)
    print("BLEU-1:", b1, "BLEU-2:", b2, "BLEU-3:", b3, "BLEU-4:", b4)
    print("ROUGE-1:", scores['rouge1'].fmeasure, "ROUGE-2:", scores['rouge2'].fmeasure,
"ROUGE-L:", scores['rougeL'].fmeasure)
    print("-"*60)

results = {
    "BLEU-1": np.mean(bleu1),
    "BLEU-2": np.mean(bleu2),
    "BLEU-3": np.mean(bleu3),
    "BLEU-4": np.mean(bleu4),
    "ROUGE-1": np.mean(rouge1),
    "ROUGE-2": np.mean(rouge2),
    "ROUGE-L": np.mean(rougel)
}

results_df = pd.DataFrame(results, index=["Score"])
print(results_df)
```

OUTPUT :

Query: Operations & Security
Reference: What is return on assets for banks?
Prediction: Operations & Security
BLEU-1: 0 BLEU-2: 0 BLEU-3: 0 BLEU-4: 0
ROUGE-1: 0.0 ROUGE-2: 0.0 ROUGE-L: 0.0

Query: General Finance
Reference: What is leverage in finance?
Prediction: Finance
BLEU-1: 0 BLEU-2: 0 BLEU-3: 0 BLEU-4: 0
ROUGE-1: 0.33333333333333337 ROUGE-2: 0.0 ROUGE-L: 0.33333333333333337

Query: Digital Banking
Reference: What is UPI Lite?
Prediction: Digital Banking
BLEU-1: 0 BLEU-2: 0 BLEU-3: 0 BLEU-4: 0
ROUGE-1: 0.0 ROUGE-2: 0.0 ROUGE-L: 0.0

Query: Basic Accounts
Reference: What is an NBFC?
Prediction: Accounts
BLEU-1: 0 BLEU-2: 0 BLEU-3: 0 BLEU-4: 0
ROUGE-1: 0.0 ROUGE-2: 0.0 ROUGE-L: 0.0

Query: Basic Accounts
Reference: What is a RuPay card?
Prediction: Accounts
BLEU-1: 0 BLEU-2: 0 BLEU-3: 0 BLEU-4: 0
ROUGE-1: 0.0 ROUGE-2: 0.0 ROUGE-L: 0.0

.
.
.
.
.

	BLEU-1	BLEU-2	BLEU-3	BLEU-4	ROUGE-1	ROUGE-2	ROUGE-L
Score	0.000896	0.000401	0.000312	0.000268	0.058135	0.001667	0.058135

Learning Outcome:

Students will be able to fine-tune, evaluate, and deploy a language model for customer support systems. They will gain practical experience in computing **BLEU and ROUGE scores**, enabling proper performance assessment of NLP models in real-world applications.